

Nextflow Pipeline Documentation

NGS Read Simulator, Mapping & Variant Calling Workflow

This documentation details how to configure, execute, and evaluate the updated, automated Nextflow implementation of the pipeline. The pipeline seamlessly combines your manual Python steps into a parallelised, reproducible workflow that manages reference genomic cleaning, synthetic mutation and read simulation, concurrent variant calling with two independent callers, VCF consolidation, metrics validation, and real-world sample scoring.

1. System Requirements & Environment Setup

The pipeline requires Nextflow (DSL2 enabled) and a Conda environment containing the required bioinformatics suites.

1.1 Install Nextflow

Ensure you have Java 11 or later installed, then download and configure Nextflow using the following execution loop:

```
curl -s https://nextflow.io | bash
chmod +x nextflow
mv nextflow /usr/local/bin/
```

1.2 Conda Environments

The pipeline orchestrates specialized tools under the hood that match your exact versioning footprint:

- **Main Environment (my_env):** Used for standard mapping and validation utilities. Contains bcftools (v1.21), samtools (v1.20), and minimap2.
- **Snippy Environment (snippy_460):** Dedicated environment to isolate older Python/tool constraints safely. Contains snippy (v4.6.0).

2. Input File Layout & Project Directory

To ensure smooth orchestration and directory resolution on the CLIMB platform, maintain the following project tree structure:

```
rp1b/
├── main.nf                # The primary Nextflow workflow orchestration
├── script
│   ├── simulate_mutations.py # Base mutation generator script
│   ├── simulate_reads.py    # Synthetic read generator script
│   └── run_bcftools_only.py  # Bcftools handling execution wrapper
```

```

├── run_snippy_only.py          # Snippy handling execution wrapper
├── merge_vcfs.py              # VCF merge processing utility
├── calculate_metrics.py        # Precision/Recall assessment engine
├── data/
│   ├── raw/
│   │   └── EcoliK12-MG1655.fasta    # Synthetic evaluation baseline reference
│   ├── sequence
│   │   └── GCF_000005845.2.fasta    # Downloaded authentic wild-type E. coli
│   ├── reference
│   │   └── processed/
│   │       └── SRR25083113_1.fastq    # Real E. coli dataset high-throughput
│   └── sequencing reads

```

3. Pipeline Configuration Parameters

Parameters can be modified seamlessly at runtime using the '--' command-line flag syntax. Default variables are directly mapped from your verified Jupyter Notebook configurations.

Parameter	Default Value	Description
--fasta	data/raw/EcoliK12-MG1655.fasta	Path to the starting baseline reference FASTA file structure.
--real_fastq	data/processed/SRR25083113_1.fastq	Path to authentic real-world E. coli sequence raw fastq file.
--real_ref	data/raw/GCF_000005845.2.fasta	Target clean reference genome matching the validation real sample properties.
--snps	300	Target number of single nucleotide polymorphisms to synthetically introduce.
--indels	10	Target number of insertion/deletion anomalies to synthetically introduce.
--depth	30	Sequencing depth alignment target threshold (e.g. 30x genome coverage).
--read_len	100	Base read window section lengths configurations (100bp window slices).
--outdir	data/processed	Destination folder path for pipeline final artifact assembly outputs.

4. Running the Pipeline

4.1 Running with Defaults

To run the full end-to-end pipeline automatically using your default validation parameters (300 SNPs, 10 INDELS, 30x depth, 100bp reads):

```
nextflow run main.nf
```

4.2 Running with Custom Validation Scale

If you want to simulate a higher variant load (e.g., 500 SNPs and 50 INDELs) with deeper coverage (50x) to test pipeline robustness:

```
nextflow run main.nf --snps 500 --indels 50 --depth 50
```

NOTE: True randomisation of variant generation is controlled internally. If you wish to disable fixed seeds and evaluate completely randomised mutation profiles across runs, remove the `set.seed(123)` call inside your `functions/make_mutations.py` dependency script.

5. Understanding the Pipeline Stages

Nextflow translates your manual, sequential command loops into highly parallelised asynchronous execution blocks:

- **SIMULATE_MUTATIONS:** Cleans the baseline FASTA file into a pure base-string structure, introduces user-defined mutations via your core Python functions, and automatically outputs both an intermediary mutated sequence and your baseline benchmark key (`ground_truth.txt`).
- **SIMULATE_READS:** Consumes the intermediary sequence file to generate short fragmented section records matching the target depth configuration.
- **ALIGN_READS / ALIGN_REAL:** Coordinates fast raw-read sorting, index mapping, and duplicate filtering using minimap2 and samtools pipelines concurrently.
- **Concurrent Variant Calling (CALL_BCFTOOLS & CALL_SNIPPY):** Nextflow runs both callers side-by-side in independent directory silos, dramatically optimising overall execution windows.
- **MERGE_VCFs:** Automatically applies compression (bgzip) and indexing (tabix) before executing structural merging algorithms to unify the divergent rows into a singular consolidated output file (`merged_samples.vcf`).
- **VALIDATE_METRICS:** Runs an automated evaluation sweep comparing caller performance (individual vs. combined models) directly against the `ground_truth.txt` blueprint.
- **SCORE_VCF_RECORDS:** Assesses caller confidence parameters in real datasets and appends a formatted accuracy trust flag inside the INFO field for simplified downstream validation filtering.

6. Reviewing Generated Outputs

Upon pipeline teardown, step into your designated output folder (`data/processed`) to find the validated evaluation outputs:

6.1 Performance Validation Reports

Open `merged_samples_metrics.txt` to monitor overall accuracy performance. The pipeline automatically calculates precision and recall based on standard statistical metrics:

- Precision (PPV) = $TP / (TP + FP)$

- Recall (Sensitivity) = $TP / (TP + FN)$

Your exact reference benchmarks can be reviewed for pipeline calibration sanity checks:

Caller Run Setup	True Positives (TP)	False Positives (FP)	False Negatives (FN)	Observed Recall	Observed Precision
bcftools_variants	299	7	11	0.9645	0.9771
ecoli_snippy.raw	296	10	14	0.9548	0.9673
merged_samples	299	7	11	0.9645	0.9771

6.2 Trust Assessment of Real Biological Variants

For the real biological E. coli sample track, inspect data/processed/real/scored_real_variants.vcf. Every validated variant row is graded with an accuracy confidence score, allowing you to quickly isolate low-confidence artifacts or mapping mismatches:

```
NC_000913.3 778165 . TTATGCTGGTCATA TTA 166.459 .
INDEL;TRUST_SCORE=1.0
```

6.3 Visual Diagnostics

To visually inspect structural anomalies like lower regional coverage bounds or problematic read overhang insertions, run samtools tview directly inside your active environment terminal:

```
samtools tview data/processed/mapped_reads.bam data/raw/EcoliK12-MG1655.fasta
```